

PSP - Oracle to Aurora PostgreSQL conversion syntax for SQLs

- [Oracle to Postgres SQL Conversion](#)
 - [Schemagen/DDL Changes](#)
 - [Query/DML changes](#)

Oracle to Postgres SQL Conversion

Oracle	Aurora PostgreSQL
alter session set current_schema=pspadm	SET search_path TO pspadm, pspapp (can specify multiple schemas)
alter session set timezone=	SET timezone=
alter session set constraints = deferred	SET CONSTRAINTS ALL DEFERRED;
Set ClientInfo: dbms_application_info.set_client_info (?); dbms_application_info.set_module(?,?); OracleDB: conn.setClientInfo("OCSID.ACTION", StringUtils.left(requestName, 64)); conn.setClientInfo("OCSID.MODULE", StringUtils.left((discoveredCompanyId + "/" + threadId), 64));	Oracle supports setting CLIENTID,ACTION,MODULE attributes separately, but Postgres has a single APPLICATION attributes, so we need to concatenate the values as: set application_name= StringUtils.left((discoveredCompanyId + ":" + threadId + ':' +requestName , 64));
NVL	COALESCE
SYS_EXTRACT_UTC(SYSTIMESTAMP)	now() at time zone 'utc'
CURRENT_DATE - modified_date	CURRENT_DATE - modified_date // if modified_date is a Date column extract(days from CURRENT_DATE - modified_date) // if modified_date is a Timestamp column
Is_No_Post = 1	Is_No_Post = '1' — strict type checking
DECODE	Rewrite using CASE
alias optional for in-line views	alias required for in-line views
outer join (+)	LEFT OUTER JOIN
rownum	row_number() OVER () ROWNUM,
Pagination using rownum where rownum <= ?) where rnum >= ?	LIMIT x OFFSET y rownum <= ? -- LIMIT rnum >= ? -- OFFSET
trunk(CURRENT_DATE)	CURRENT_DATE — returns date without time
sysdate - 7	CURRENT_TIMESTAMP - INTERVAL '7 DAY'
Sysdate -1/24	CURRENT_TIMESTAMP - INTERVAL '1 HOUR'

Concatenation in Oracle Max(pmt_headers.tx_date) ',' Max(pmt_headers. pmt_disposition_date) ',' Max(deposit.tx_date)	Should be done in Postgres as - CONCAT(Max(pmt_headers.tx_date),',', Max(pmt_headers.pmt_disposition_date),',', Max(deposit. tx_date)) With ' ' if one of the values is null , then whole string will be returned as null
Date column in WHERE clause Last_modify_date >= To_date('02/08/2020', 'MM/DD/YYYY') Last_modify_date >= To_date('02/08/2020 00:00:00', 'MM/DD/YYYY hh24:mi:ss')	Last_modify_date >= date '2020-02-08' Last_modify_date >= timestamp '2020-02-08 00:00:00'
List Aggregation	

Schemagen/DDl Changes

Feature	Oracle	Postgres
Table Partitioning	create table xx () partition by hash (company_id) partitions <NUM-PARTITIONS>;	TBD
Index Partitioning	CREATE INDEX <indexName> ON <tableName> (col1,...) local online;	TBD
Synonyms	CREATE PUBLIC SYNONYM PSP_HCM401 K_COMPANY_POLICY FOR pspadm.PSP_HCM401K_COMPANY_ POLICY;	No need for synonyms. Application uses SET SEARCH_PATH pspadm
Sequences	- create sequence <sequenceName> START WITH <START-VALUE> INCREMENT BY <INCR-VALUE>; - Mysequence.nextval;	- create sequence <sequenceName> START WITH 1 INCREMENT BY 1; - nextval('mysequence');
Triggers & Stored Procs	Oracle specific SQL in all.dict using "sqltextblock"	Dialect specific sqltextblock
PL/SQL (Oracle) to PL/pgSQL (Postgres)	Current pl/sql code	https://www.postgresql.org/docs/13/plpgsql-porting.html NO_DATA_FOUND and TOO_MANY_ROWS These exceptions are disabled by default for selects in PLpgSQL. You need to add keyword STRICT after any keyword INTO in all selects, when you need to keep single row checking in stored PLpgSQL code. See documentation.
Packages	Supported	Not supported. But this can be achieved by grouping all procedures into schema.

Function Based Index	eg CREATE INDEX "PSPADM"."PSP_LEDGER_BALANCE_U2" ON "PSPADM"."PSP_LEDGER_BALANCE" ("COMPANY_FK", "LEDGER_ACCOUNT_FK", TRUNC ("BALANCE_DATE"))	CREATE INDEX CONCURRENTLY psp_ledger_balance_u2 ON pspadm.psp_ledger_balance USING BTREE (company_fk ASC, ledger_account_fk ASC, DATE(balance_date) ASC);
Trigger	Trigger definition contains triggering event as well as code.	It is achieved by create trigger DDL for event and

Query/DML changes

DML	Oracle	Postgres
DELETE	DELETE mytable WHERE column_name = 'column_amount';	DELETE FROM table_name WHERE column_name = 'column_amount';
Autonomous transactions	Supported	Not supported. Workaround: DBLINK feature allows the session to connect and query remote PostgreSQL servers or we can also open a new connection to same PostgreSQL server locally. This feature comes handy for achieving Autonomous Transactions in PostgreSQL.
Subquery in FROM		PostgreSQL requires a sub-SELECT surrounded by parentheses, and an alias must be provided for it. The alias is not mandatory for Oracle. SELECT * FROM (SELECT * FROM table_a) AS foo